内核数据分析

1. ptrace

`strace` 能够拦截每一个系统调用，靠的就是 Linux 内核提供的 `ptrace` (process trace) 系统调用。

% time	seconds	usecs/call	calls	errors	syscall
78.53	108.165195	879	123015	16651	futex
9.00	12.402343	1194	10386		pselect6
7.10	9.784233	227540	43	6	wait4
2.07	2.850116	2	1157864	145	read
1.95	2.686470	1416	1897	3	ppoll
0.90	1.235197	34	35651		write
0.13	0.183176	1	96995	4297	newfstatat
0.08	0.105386	7	14507	9221	ioctl
0.05	0.063918	9	6631		getpid
0.03	0.044712	2	16708	4109	openat
0.02	0.033998	2	12888		close
0.02	0.030877	25	1225		brk
0.02	0.027061	16	1610		munmap
0.01	0.019909	48	407		unlinkat
0.01	0.017909	8	2128	10	sendto
0.01	0.010734	3	3495		mmap
0.01	0.010724	0	18803	5	lseek
0.01	0.008912	3	2686		mprotect
0.01	0.008472	0	13046		fstat
0.01	0.008396	139	60		recvfrom
0.01	0.007112	2	2690		getdents64
100.00	137.729353	89	1530356	35404	total

1. futex (78.53% - 核心瓶颈)

- 来源：多线程同步与锁竞争。
  - 等待慢线程：主循环（推理线程）执行速度快，但在等待相机线程填充视频帧缓冲区，或者等待底层控制器（USB 串口）返回数据。

- GIL 锁争用：由于 Python 存在全局解释器锁（GIL），当多个线程（如两个相机 + 一个控制 + 一个推理）同时运行时，内核会频繁调用 futex 来挂起和唤醒这些抢夺 CPU 权的线程。

2. pselect6 (9.00%)

- 来源：**多媒体/网络设备**的**超时等待**。
- 解释：它用于监视多个文件描述符的状态。
- 视频流捕获：在使用 **OpenCV (V4L2)** 时， `pselect6` 负责检查摄像头是否有新的图像数据到达。
- 特点：如果该耗时较高，说明 CPU 在“等待硬件”。例如摄像头设为** 30fps**，CPU 就要每隔 **33ms** 确认一次数据是否准备好。

3. wait4 (7.10%)

- 来源：子进程状态监控。
- 解释：这通常出现在**多进程**架构中。
- 主进程管理：LeRobot 可能会 fork 出独立的子进程来处理视频解码或数据记录。**主进程通过 wait4 持续监控子进程是否崩溃或结束**。
- 异常点：单次调用耗时（usecs/call）高达 22.7ms，这在多进程同步中属于典型的阻塞等待。

4. read (2.07% - **次数惊人**)

- 通过后面知道这是在读取数据，都是预加载。

5. ppoll (1.95%)

- 来源：事件监听（如输入设备或网络）。
  — 解释：类似于 pselect6，但在 Python 的多线程/异步库中更常用。
- 消息传递：可能是进程间通信（IPC）或者监听来自 VNC/键盘的输入事件。它代表了程序在空闲时段的“挂起”状态。

6. write (0.90%)

- 来源：指令发送与日志记录。
- 控制指令：向 /dev/ttyACM0 发送**电机运动指令**。
- 数据存盘：**将采集到的图像和状态写入磁盘**（Dataset Recording）。
- 分析：3.5 万次调用相对于 115 万次 read 显得很健康，说明指令发送的频率比状态读取的频率要低得多，控制频率比较稳定。

2. time

/usr/bin/time -v 通过内核的 `wait4` 或 getrusage 在进程结束时获取汇总的统计数据。

User Time (用户态时间)：CPU 执行你的代码（如 PyTorch 推理、OpenCV 矩阵运算）的时间。

System Time (内核态时间)：CPU 代替你的进程在内核中跑指令（如驱动读写、网络收发、分配内存）的时间。

```
User time (seconds): 71.97
System time (seconds): 20.12
Percent of CPU this job got: 152%
Elapsed (wall clock) time (h:mm:ss or m:ss): 1:00.50
Maximum resident set size (kbytes): 925096
Minor (reclaiming a frame) page faults: 242026
Voluntary context switches: 5342373
Involuntary context switches: 621151
File system inputs: 1248
File system outputs: 304880
```

这说明在 `1:00.50 (60.5秒)` 的运行时间内，CPU 的总执行时间是 71.97s `(User)+20.12s(System)=92.09s` 。 公式：`92.09/60.5 ≈ 152%`。 这证明树莓派 4 个核心里，平均有 `1.52` 个核心 一直在全速运转。

这俩数据是一次性测得的。

之所以 time 命令报告的总 CPU 时间（92.09s）和 strace 报告的系统调用总时间（137s）不一致，甚至 strace 的统计时间超过了进程运行的真实时长，主要由以下三个核心原因造成：

1. 多线程的时间叠加效应（Wall-clock vs. Sum of Threads）
   这是最根本的原因。strace -f 参数会追踪所有子进程和线程。如果在某一秒内，你的机器人有 4 个线程同时执行了系统调用，strace 会把这 4 个线程的系统调用时间全部相加。
2. 数据来源的定义不同

这两个工具对“时间”的统计口径存在本质区别：
time (92.09s)：它是通过内核的 `getrusage` 获取的，记录的是 CPU 真正执行指令 的时间（不包括进程被挂起、等待 I/O 或等待 strace 响应时的闲置时间）。
strace (137s)：它记录的是系统调用的周转时间（Elapsed time of syscalls）。如果一个线程调用 `futex` 睡眠了，虽然它不占用 CPU 指令，但 `strace` 依然会记录它“占用”了系统调用时间，直到它被唤醒。

后面我把 `strace` 关掉

```
User time (seconds): 76.40
System time (seconds): 2.20
Percent of CPU this job got: 183%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:42.78
```

系统调用时间量减少巨大。

3. perf

perf 工作在内核硬件计数器层面，开销极低，能告诉我们 CPU 到底在执行哪些函数，而不是像 strace 那样只停留在系统调用表面。

4.避免冷启动+strace

```
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 79.81    5.891580         119     49505      7097 futex
 16.32    1.204536         421      2857           pselect6
  1.46    0.107787          38      2821           write
  0.99    0.073202       14640         5         5 restart_syscall
  0.46    0.034186          14      2280       842 ioctl
  0.31    0.022797          23       974           getpid
  0.18    0.013190         599        22           brk
  0.12    0.008969          35       253       253 newfstatat
  0.12    0.008761          34       254           fstat
  0.10    0.007518          29       254           openat
  0.07    0.005027           4      1018           lseek
  0.05    0.003708           2      1523           read
  0.01    0.000380           1       255           close
  0.00    0.000000           0         6           mprotect
```

1. `futex` (79.81%)：多核系统

- 全称 Fast Userspace Mutex。它是 Linux 内核用来协调多个线程同步的机制。
- **为什么这么多**：有 27 个线程在抢 4 个核心。当线程 A 想要处理图像，但发现数据被线程 B 锁住了，或者 CPU 核心正被线程 C 占着，线程 A 就会发起 `futex` 调用，告诉内核先停会有资源了再来。
- **信号意义**：这是系统**内耗**的最大来源，多核环境下严重的资源争夺。

- 因为神经网络推理是分块执行的（算一层，停一下，再算下一层），**每当一层计算结束，线程就会去同步数据**。4 个核心被这些矩阵运算塞满了，每次同步都会触发一次 futex 来重新排队。

2. `pselect6` (16.32% 1.204536s)：硬件数据

- 这是一个多路 I/O 复用器。它让进程挂起，直到某个设备（比如摄像头）准备好数据。
- **摄像头**在以 30fps 运行。每隔约 33ms，系统就要通过 `pselect6` 检查一次摄像头出图了没。
- **信号意义**：421 微秒的平均耗时说明系统在等待硬件响应。由于它占了 16%，说明 CPU 有相当一部分时间是在等数据，这是实时控制中无法避免的硬开销。

3. `write` (1.46%)

- 把算好的动作指令（比如 6 个电机的角度）通过 USB 串口发给驱动板。
- **信号意义**：这是你**控制频率**的证据。每秒约 280 次调用，说明指令发得很勤快，且每次只花 38 微秒（极快），说明系统底层的 USB 总线发送非常顺畅，完全没有堵塞。
- 读视频和写指令的发送次数差不多。

4. `restart_syscall` (0.99%)

- 当一个系统调用（比如正在等的 `pselect6` ）被某个信号中断后，内核自动重新启动该调用的机制。
- **为什么会有**：在极高负载下，或者当你挂载 `strace` 追踪时，系统会产生一些信号中断当前的阻塞操作。

5. `ioctl` (0.46%)

- Input/Output Control，用于执行设备特有的操作（比如调整摄像头曝光、请求特定的视频帧缓冲区）。这是在跟硬件做**底层对齐**。

6. read 减少巨大 (从 116万次 降到 1523次)

- 现象：之前数据中占据统治地位的 read 系统调用，在纯推理阶段几乎看不到了。
- 原因：之前那 110 万次 read 全是启动时加载模型文件（PyTorch 加载几百 MB 的权重）产生的。在真正的机器人控制循环里，读取电机反馈的开销极低，系统在这方面非常健康。

```
  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
15843 vel       20   0 5476552 833296 215800 R 357.8  20.4   0:39.58 python
```

主要瓶颈还是推理的时候CPU占用。